Agenda:

1. Single Number with Sorted Array
2. Search in Rotated Sorted Array
3. Painter's Partition - Find min count of painters
4. Square root

Q1: Given an integer array A[ ], where every element occurs **twice** except for one, find the unique element. **Note:** Duplicate elements are **adjacent** to each other.

A[ ] =

0	1	2	3	4	5	6	7	8	9	10
7	7	6	6	3	8	8	1	1	9	9

A[ ] =

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Elements	3	3	1	1	8	8	10	10	19	6	6	2	2	4	4

Sol 1 → Ans = $\forall i, \wedge A[i]$ TC = O(N) SC = O(1)

Sol 2 → 1) Search space → $l = 0$ $r = N-1$

2) check if mid is answer → (A[mid] != A[mid-1] &&

A[mid] != A[mid+1])

3) Go left or right →

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Elements	3	3	1	1	8	8	10	10	19	6	6	2	2	4	4

(even odd) index (odd even) index

```
l = 0        r = N-1
while (l <= r) {
    mid = (l + r) / 2
    if ((mid == 0 || A[mid-1] != A[mid]) &&
        (mid == N-1 || A[mid+1] != A[mid]))
            return A[mid]
    if (mid == 0 || A[mid-1] != A[mid]) {   // A[mid] = A[mid+1]
        if (mid % 2 == 0)   l = mid + 2   // even - odd
        else                r = mid - 1   // odd - even
    } else {                              // A[mid] = A[mid-1]
```

```
if ( mid % 2 == 1 )    l = mid + 1        // even - odd
else                   r = mid - 2        // odd - even
}
}
```

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Elements	3	3	1	1	8	8	10	10	19	6	6	2	2	4	4

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Elements	3	3	1	1	8	8	10	10	19	6	6	2	2	4	4

elements on left

from first occurrance

even ⇒ go right

equal pair on left

odd ⇒ go left

equal pairs + unique element

TC = $O(\log_2(N))$ SC = $O(1)$

Q2: Imagine you have an array that was **sorted** initially, but someone rotated it at an **unknown** index. The resulting array is a **rotated sorted array**. The challenge is to find index of a specific element in this rotated array. If the element is not present, return -1.

	0	1	2	3	4	5	6	7	8
A =	4,	5,	6,	7,	8,	9,	1,	2,	3]

K = 7 Ans = 3

1 3 8 10 12 15 16 18 20

3 times

16 18 20 1 3 8 10 12 15

part 1 part 2

16	18	20	1	3	8	10	12	15

$\forall i$ A[i] in part 1 >= $\forall j$ A[j] in part 2

if (x >= A[0])

⇒ x is in part 1

else ⇒ x is in part 2

```
l = 0        r = N-1
while (l <= r) {
    mid = (l+r) / 2
    if (A[mid] == K) return mid
```

```
if ( A[mid] >= A[0]) {      // mid is in part 1
    if ( K >= A[0]) {       // K is in part 1
        if ( K < A[mid])    r = mid - 1
        else                l = mid + 1
    } else                  l = mid + 1     // K is in part 2
} else {                    // mid is in part 2
    if ( K >= A[0])         r = m - 1       // K is in part 1
    else {                  // K is in part 2
        if ( K < A[mid])    r = mid - 1
        else                l = mid + 1
    }
}
}
```

$TC = O(\log(N))$ $SC = O(1)$

l	m		r	~~m~~				~~r~~
0	1	2	3	4	5	6	7	8
16	18	20	1	3	8	10	12	15

K = 18

~~l~~				~~m~~	~~l~~	~~m~~	m, l	r
0	1	2	3	4	5	6	7	8
16	18	20	1	3	8	10	12	15

K = 12

~~l~~			~~m~~	r, m, l	~~m~~	~~r~~
0	1	2	3	4	5	6
4	5	6	7	0	1	2

K = 0

Painter's Partition Problem

Given N boards with length of each board

a) A painter takes 1 unit of time to paint 1 unit of length.

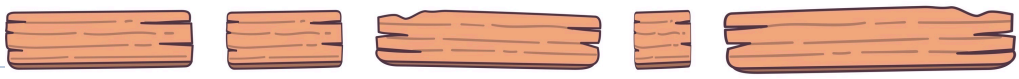
b) A board can only be painted by 1 painter.

c) A painter can only paint boards placed next to each other (i.e continuous segment).

Q3: Find the **minimum count of painters** required to paint all the boards in **T unit of time.**

N = **5**

T = **10**



5 → 3 → 6 → 1 → 9

p1: 5, 2 (10)

p2: 4, 3 (10)

p3: 1 (10)

A = [5 8 2 6 1 3 9 10 7] T = 12

p1 (12): 5 → 7 → 4

p2 (12): 2 → 6 → 5

p3 (12): 2 → 3

p4 (12): 2

p5 (12): 3

p6 (12): 2 → 5

Ans = 6

```
ans = 1        time = T
for i → 0 to (N-1) {
    if ( A[i] > T )    return -1    // not possible
    if ( time >= A[i] )    time = time - A[i]
```

```
        else {  ans++
                time = T - A[i]
            }
    }

return ans
```

TC = $\underline{O(N)}$ SC = $\underline{O(1)}$

Q4: Find the integer part of **square root** of the given integer **N** without using in-built functions.

Example: N = 50 Ans = 7 N = 16 Ans = 4 N > 0

N = 20 Ans = 4

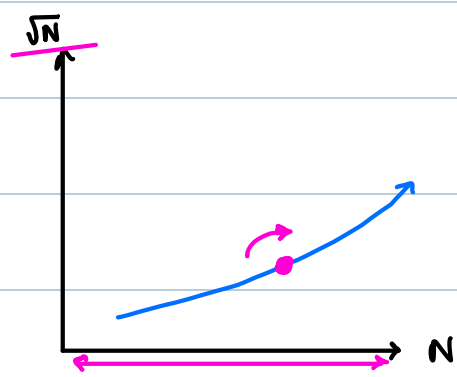
$\boxed{1 <= \sqrt{N} <= N}$ → search space

$\boxed{x^2 <= N \ \&\& \ (x+1)^2 > N}$ → x is ans for N

```
l = 1        r = N
while (l <= r) {
    mid = l + (r - l)/2
    if (mid * mid <= N && (mid+1) * (mid+1) > N)
        return mid
    if ( mid * mid < N )    l = mid+1
    else    r = mid - 1
}
```

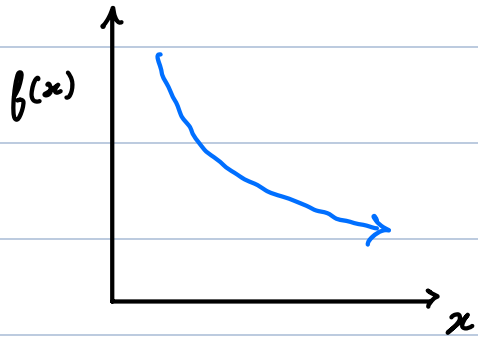
TC = $O(\log_2(N))$ SC = $O(1)$

$\sqrt{N} \propto N$



Monotonic increasing

$f(x) \propto \frac{1}{x}$



Monotonic decreasing

The following code used to calculate Unique Element but it sometimes throws ArrayIndexOutOfBoundsException. Which fix correctly removes the bug?

```
if (arr[mid] != arr[mid - 1] && arr[mid] != arr[mid + 1]) {
    return arr[mid];
}
```

((mid == 0 || A[mid-1] != A[mid]) &&

(mid == N-1 || A[mid+1] != A[mid]))

What will be the output for following code snippet?

```
public class Test4 {
    public static void main(String[] args) {
        System.out.println(sqrt(49));  → 7
    }

    static int sqrt(int n) {
        int lo = 1, hi = n, ans = 0;
        while (lo <= hi) {
            int mid = lo + (hi - lo)/2;

            if (mid * mid <= n) {
                ans = mid;
                lo = mid + 1;
            } else hi = mid - 1;
        }
        return ans;
    }
}
```